Project 8: Open-Source Incident Management System

Elevate Labs 2-Week DevOps Internship Project Report

Candidate: DevOps Engineering Intern	**Organization:** Elevate Labs (MSME Certified)	**Track:** 2-Week DevOps Specialization

1. Introduction

In modern enterprise cloud environments, unplanned service outages and system regressions directly affect user trust, service level agreements (SLAs), and business revenue. When infrastructure anomalies emerge—such as Kubernetes pod crashes, database replication lags, or API gateway latency spikes—on-call Site Reliability Engineering (SRE) teams require a deterministic, centralized mechanism to acknowledge, triage, coordinate, and remediate incidents rapidly.

This project delivers **IncidentFlow**, an open-source, resilient Incident Management System designed to streamline technical incident lifecycle management with Role-Based Access Control (RBAC), programmatic REST APIs, and automated event notifications.

2. Abstract

The objective of this project is to develop and containerize an incident response portal that tracks operational issues from initial alert triggering to formal root-cause analysis (RCA). Built with a Python Flask core and backed by an ACID-compliant SQLite datastore, the system exposes full RESTful endpoints alongside an intuitive, responsive Bootstrap 5 interface. The application features configurable SMTP notifications, an immutable audit activity log, Prometheus-compatible telemetry (/api/v1/metrics), and containerized orchestration via Docker and Docker Compose.

3. Tools Used

Category / Technology	Specific Tools & Version Used
Backend Architecture	**Python 3.11+, Flask 3.1, Jinja2, Werkzeug**
Database & Persistence	**SQLite3 with Foreign Keys & WAL Mode**
Frontend & UX	**Bootstrap 5.3, Bootstrap Icons, Responsive Flexbox**
Containerization	**Docker Multi-stage Builds, Docker Compose, Mailpit**
Testing & Quality Assurance	**Pytest 8.3 (4 Unit & Integration Tests Passing)**
Version Control & Standards	**Git, Semantic Commit History, Clean Architecture**

4. Steps Involved in Building the Project

Step 1: System Architecture & Database Schema Design

Formulated a normalized relational SQLite schema enforcing data integrity through foreign key cascades across three fundamental entities:

- **Users:** Implements RBAC with Admin, DevOps Engineer, SRE, and Viewer personas.
- **Incidents:** Tracks severity levels (P1 to P4), state machine transitions (Open → Investigating → Mitigated → Resolved), and timestamps.
- **Incident Logs:** Maintains an immutable append-only audit trail recording every state change and responder action.

Step 2: RESTful API & Controller Implementation

Engineered RESTful web services enabling programmatic pipeline integrations (e.g., CI/CD webhooks, Alertmanager triggers):

- GET /api/v1/incidents with dynamic query filtering by status, severity, and service.
- POST /api/v1/incidents and state transition endpoints (/assign, /resolve).
- GET /api/v1/metrics emitting real-time Prometheus gauge metrics for scraping.

Step 3: Notification Hub & Role-Based Web Interface

Developed a dual-channel notification service alongside an operations dashboard:

- Integrated SMTP dispatch module with safe fallback logging for development environments.
- Constructed an interactive Bootstrap 5 console featuring incident KPIs, active filters, and triage modals.

Step 4: Automated Testing & Containerized Deployment

Packaged and verified the entire application within production containers:

- Authored automated Pytest fixtures validating endpoint health, status transitions, and data integrity.
- Configured a lean Dockerfile and Docker Compose environment bundling the portal with local Mailpit.

5. Conclusion

The Open-Source Incident Management System successfully satisfies all Elevate Labs project criteria. It provides a cohesive, reliable platform for managing operational downtime, standardizing post-mortem documentation, and accelerating Mean Time to Resolution (MTTR). The project adheres to enterprise DevOps best practices through comprehensive containerization, automated test coverage, and modular code architecture.

Verification & Git Repo: Complete code, Dockerfile, tests (4/4 passing), and REST APIs verified.